

# LAB PROGRAMME

## CHAPTER 1 : INTRODUCTION

NAME: S. BHUVANESH

REG NO: 192521004

1. Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.

Example 1:

Input: words = ["abc","car","ada","racecar","cool"]

Output: "ada"

Explanation: The first string that is palindromic is "ada".

Note that "racecar" is also palindromic, but it is not the first.

Example 2:

Input: words = ["notapalindrome","racecar"]

Output: "racecar"

Explanation: The first and only string that is palindromic is "racecar".

```
main.c
1  #include <stdio.h>
2  #include <string.h>
3  #include <stdbool.h>
4
5  bool isPalindrome(const char* str) {
6      int len = strlen(str);
7      for (int i = 0; i < len / 2; i++) {
8          if (str[i] != str[len - 1 - i]) return false;
9      }
10     return true;
11 }
12
13 const char* firstPalindrome(const char** words, int wordsSize) {
14     for (int i = 0; i < wordsSize; i++) {
15         if (isPalindrome(words[i])) return words[i];
16     }
17     return "";
18 }
19
20 int main() {
21     const char* words1[] = {"abc", "car", "ada", "racecar", "cool"};
22     printf("Result 1: %s\n", firstPalindrome(words1, 5)); // Output: ada
23
24     const char* words2[] = {"notapalindrome", "racecar"};
25     printf("Result 2: %s\n", firstPalindrome(words2, 2)); // Output: racecar
26     return 0;
27 }
28
29
```

input

Result 1: ada  
Result 2: racecar

2. You are given two integer arrays `nums1` and `nums2` of sizes `n` and `m`, respectively. Calculate the following values: `answer1` : the number of indices `i` such that `nums1[i]` exists in `nums2`. `answer2` : the number of indices `i` such that `nums2[i]` exists in `nums1`. Return `[answer1, answer2]`.

Example 1:

Input: `nums1 = [2,3,2]`, `nums2 = [1,2]`

Output: `[2,1]`

Explanation:

Example 2:

Input: `nums1 = [4,3,2,3,1]`, `nums2 = [2,2,5,2,3,6]`

Output: `[3,4]`

Explanation:

The elements at indices 1, 2, and 3 in `nums1` exist in `nums2` as well. So `answer1` is 3.

The elements at indices 0, 1, 3, and 4 in `nums2` exist in `nums1`. So `answer2` is 4.

```
main.c
1  #include <stdio.h>
2  #include <stdbool.h>
3
4  void countCommon(int* nums1, int n1, int* nums2, int n2, int* ans1, int* ans2) {
5      *ans1 = 0;
6      *ans2 = 0;
7      for (int i = 0; i < n1; i++) {
8          for (int j = 0; j < n2; j++) {
9              if (nums1[i] == nums2[j]) { (*ans1)++; break; }
10         }
11     }
12     for (int i = 0; i < n2; i++) {
13         for (int j = 0; j < n1; j++) {
14             if (nums2[i] == nums1[j]) { (*ans2)++; break; }
15         }
16     }
17 }
18
19 int main() {
20     int nums1[] = {4, 3, 2, 3, 1};
21     int nums2[] = {2, 2, 5, 2, 3, 6};
22     int ans1, ans2;
23     countCommon(nums1, 5, nums2, 6, &ans1, &ans2);
24     printf("Output: [%d, %d]\n", ans1, ans2); // Output: [3, 4]
25     return 0;
26 }
27
28
29
```

input

Output: [3, 4]

3. You are given a 0-indexed integer array `nums`. The distinct count of a subarray of `nums` is defined as: Let `nums[i..j]` be a subarray of `nums` consisting of all the indices from `i` to `j` such that  $0 \leq i \leq j < \text{nums.length}$ . Then the number of distinct values in `nums[i..j]` is called the distinct count of `nums[i..j]`. Return the sum of the squares of distinct counts of all subarrays of `nums`. A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [1,2,1]`

Output: 15

Explanation: Six possible subarrays are:

[1]: 1 distinct value

[2]: 1 distinct value

[1]: 1 distinct value

[1,2]: 2 distinct values

[2,1]: 2 distinct values

[1,2,1]: 2 distinct values

The sum of the squares of the distinct counts in all subarrays is equal to  $1^2 + 1^2 + 1^2 + 2^2 + 2^2 + 2^2 = 15$ .

Example 2:

Input: `nums = [1,1]`

Output: 3

Explanation: Three possible subarrays are:

[1]: 1 distinct value

[1]: 1 distinct value

[1,1]: 1 distinct value

The sum of the squares of the distinct counts in all subarrays is equal to  $1^2 + 1^2 + 1^2 = 3$ .

```
main.c
1  #include <stdio.h>
2  #include <string.h>
3
4  int sumCounts(int* nums, int numsSize) {
5      int totalSum = 0;
6      for (int i = 0; i < numsSize; i++) {
7          int seen[1001] = {0};
8          int distinctCount = 0;
9          for (int j = i; j < numsSize; j++) {
10             if (seen[nums[j]] == 0) {
11                 seen[nums[j]] = 1;
12                 distinctCount++;
13             }
14             totalSum += (distinctCount * distinctCount);
15         }
16     }
17     return totalSum;
18 }
19
20 int main() {
21     int nums[] = {1, 2, 1};
22     printf("Output: %d\n", sumCounts(nums, 3)); // Output: 15
23     return 0;
24 }
25
26
27
28
```

Output: 15

4. Given a 0-indexed integer array `nums` of length `n` and an integer `k`, return *the number of pairs*  $(i, j)$  where  $0 \leq i < j < n$ , such that `nums[i] == nums[j]` and  $(i * j)$  is divisible by `k`.

Example 1:

Input: `nums = [3,1,2,2,2,1,3]`, `k = 2`

Output: 4

Explanation:

There are 4 pairs that meet all the requirements:

- `nums[0] == nums[6]`, and  $0 * 6 == 0$ , which is divisible by 2.
- `nums[2] == nums[3]`, and  $2 * 3 == 6$ , which is divisible by 2.
- `nums[2] == nums[4]`, and  $2 * 4 == 8$ , which is divisible by 2.
- `nums[3] == nums[4]`, and  $3 * 4 == 12$ , which is divisible by 2.

Example 2:

Input: `nums = [1,2,3,4]`, `k = 1`

Output: 0

Explanation: Since no value in `nums` is repeated, there are no pairs  $(i, j)$  that meet all the requirements.

```
main.c
1  #include <stdio.h>
2
3  int countPairs(int* nums, int numsSize, int k) {
4      int count = 0;
5      for (int i = 0; i < numsSize; i++) {
6          for (int j = i + 1; j < numsSize; j++) {
7              if (nums[i] == nums[j] && (i * j) % k == 0) {
8                  count++;
9              }
10         }
11     }
12     return count;
13 }
14
15 int main() {
16     int nums[] = {3, 1, 2, 2, 2, 1, 3};
17     printf("Output: %d\n", countPairs(nums, 7, 2)); // Output: 4
18     return 0;
19 }
20
```

input

Output: 4

5. Write a program FOR THE BELOW TEST CASES with least time complexity

Test Cases: -

- 1) Input: {1, 2, 3, 4, 5} Expected Output: 5
- 2) Input: {7, 7, 7, 7, 7} Expected Output: 7
- 3) Input: {-10, 2, 3, -4, 5} Expected Output: 5

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int parseAndFindMax(const char** arr, int size) {
5     int maxVal = -2147483648;
6     for (int i = 0; i < size; i++) {
7         char cleanStr[32] = {0};
8         int idx = 0;
9         for (int j = 0; arr[i][j] != '\0'; j++) {
10             if ((arr[i][j] >= '0' && arr[i][j] <= '9') || arr[i][j] == '-') {
11                 cleanStr[idx++] = arr[i][j];
12             }
13         }
14         if (idx > 0) {
15             int val = atoi(cleanStr);
16             if (val > maxVal) maxVal = val;
17         }
18     }
19     return maxVal;
20 }
21
22 int main() {
23     const char* input[] = {"-10", "2", "3", "\"-4,\"", "5"};
24     printf("Max Output: %d\n", parseAndFindMax(input, 5)); // Output: 5
25     return 0;
26 }
27
```

input

Max Output: 5

6. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

Test Cases

1. Empty List
  1. Input: []
  2. Expected Output: None or an appropriate message indicating that the list is empty.
2. Single Element List
  1. Input: [5]
  2. Expected Output: 5
3. All Elements are the Same
  1. Input: [3, 3, 3, 3, 3]
  2. Expected Output: 3

```
main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int compare(const void* a, const void* b) {
5      return (*(int*)a - *(int*)b);
6  }
7
8  void sortAndFindMax(int* arr, int size) {
9      if (size == 0) {
10         printf("List is empty\n");
11         return;
12     }
13     qsort(arr, size, sizeof(int), compare);
14     printf("Max element: %d\n", arr[size - 1]);
15 }
16
17 int main() {
18     int arr[] = {3, 3, 3, 3, 3};
19     sortAndFindMax(arr, 5); // Output: 3
20     return 0;
21 }
22
```

Max element: 3

7. Write a program that takes an input list of  $n$  numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

Test Cases

Some Duplicate Elements

- Input: [3, 7, 3, 5, 2, 5, 9, 2]
- Expected Output: [3, 7, 5, 2, 9] (Order may vary based on the algorithm used)

Negative and Positive Numbers

- Input: [-1, 2, -1, 3, 2, -2]
- Expected Output: [-1, 2, 3, -2] (Order may vary)

List with Large Numbers

- Input: [1000000, 999999, 1000000]
- Expected Output: [1000000, 999999]

```
main.c
1 #include <stdio.h>
2 #include <stdbool.h>
3
4 int getUnique(int* arr, int size, int* outUnique) {
5     int uniqCount = 0;
6     for (int i = 0; i < size; i++) {
7         bool isDup = false;
8         for (int j = 0; j < uniqCount; j++) {
9             if (arr[i] == outUnique[j]) { isDup = true; break; }
10        }
11        if (!isDup) {
12            outUnique[uniqCount++] = arr[i];
13        }
14    }
15    return uniqCount;
16 }
17
18 int main() {
19     int original[] = {3, 7, 3, 5, 2, 5, 9, 2};
20     int trackingBuffer[8];
21     int size = getUnique(original, 8, trackingBuffer);
22
23     printf("Unique Output Array: [ ");
24     for (int i = 0; i < size; i++) printf("%d ", trackingBuffer[i]);
25     printf("\n"); // Output: [3, 7, 5, 2, 9]
26     return 0;
27 }
28
```

input

Unique Output Array: [ 3 7 5 2 9 ]

8. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

```
main.c
1 #include <stdio.h>
2 #include <stdbool.h>
3
4 void bubbleSort(int* arr, int n) {
5     for (int i = 0; i < n - 1; i++) {
6         bool swapped = false;
7         for (int j = 0; j < n - i - 1; j++) {
8             if (arr[j] > arr[j + 1]) {
9                 int temp = arr[j];
10                arr[j] = arr[j + 1];
11                arr[j + 1] = temp;
12                swapped = true;
13            }
14        }
15        if (!swapped) break;
16    }
17 }
18
19 int main() {
20     int arr[] = {5, 1, 4, 2, 8};
21     bubbleSort(arr, 5);
22     for (int i = 0; i < 5; i++) printf("%d ", arr[i]); // Output: 1 2 4 5 8
23     return 0;
24 }
25
```

input

1 2 4 5 8

9. Checks if a given number  $x$  exists in a sorted array  $arr$  using binary search. Analyze its time complexity using Big-O notation.

Test Case:

Example  $X = \{ 3, 4, 6, -9, 10, 8, 9, 30 \}$  KEY=10

Output: Element 10 is found at position 5

Example  $X = \{ 3, 4, 6, -9, 10, 8, 9, 30 \}$  KEY=100

Output : Element 100 is not found

```
main.c
1  #include <stdio.h>
2
3  void binarySearch(int* arr, int size, int key) {
4      int low = 0, high = size - 1;
5      while (low <= high) {
6          int mid = low + (high - low) / 2;
7          if (arr[mid] == key) {
8              printf("Element %d is found at position %d\n", key, mid);
9              return;
10         } else if (arr[mid] < key) {
11             low = mid + 1;
12         } else {
13             high = mid - 1;
14         }
15     }
16     printf("Element %d is not found\n", key);
17 }
18
19 int main() {
20     // Array sorted beforehand for execution validation
21     int arr[] = {-9, 3, 4, 6, 8, 9, 10, 30};
22     binarySearch(arr, 8, 10); // Found at position 6 (0-indexed)
23     binarySearch(arr, 8, 100); // Not found
24     return 0;
25 }
26
```

input

```
Element 10 is found at position 6
Element 100 is not found
```

10. Given an array of integers  $nums$ , sort the array in ascending order and return it. You must solve the problem without using any built-in functions in  $O(n \log(n))$  time complexity and with the smallest space complexity possible.

```

main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void merge(int* arr, int l, int m, int r) {
5      int n1 = m - l + 1;
6      int n2 = r - m;
7      int* L = (int*)malloc(n1 * sizeof(int));
8      int* R = (int*)malloc(n2 * sizeof(int));
9      for (int i = 0; i < n1; i++) L[i] = arr[l + i];
10     for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
11
12     int i = 0, j = 0, k = l;
13     while (i < n1 && j < n2) {
14         if (L[i] <= R[j]) arr[k++] = L[i++];
15         else arr[k++] = R[j++];
16     }
17     while (i < n1) arr[k++] = L[i++];
18     while (j < n2) arr[k++] = R[j++];
19     free(L); free(R);
20 }
21
22 void mergeSort(int* arr, int l, int r) {
23     if (l < r) {
24         int m = l + (r - l) / 2;
25         mergeSort(arr, l, m);
26         mergeSort(arr, m + 1, r);
27         merge(arr, l, m, r);
28     }
29 }
30
31 int main() {
32     int arr[] = {38, 27, 43, 3, 9, 82, 10};
33     mergeSort(arr, 0, 6);
34     for (int i = 0; i < 7; i++) printf("%d ", arr[i]); // Output: 3 9 10 27 38 43 82
35     return 0;
36 }

```

OUTPUT:

```

3 9 10 27 38 43 82

```

11. Given an  $m \times n$  grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly  $N$  steps.

Example:

|                                  |            |
|----------------------------------|------------|
| Input: $m=2, n=2, N=2, i=0, j=0$ | Output: 6  |
| Input: $m=1, n=3, N=3, i=0, j=1$ | Output: 12 |

```

main.c
1  #include <stdio.h>
2  #include <string.h>
3
4  int findPaths(int m, int n, int maxMove, int startRow, int startColumn) {
5      long long MOD = 1000000007;
6      long long dp[50][50] = {0};
7      dp[startRow][startColumn] = 1;
8      long long count = 0;
9
10     for (int moves = 1; moves <= maxMove; moves++) {
11         long long nextDp[50][50] = {0};
12         for (int r = 0; r < m; r++) {
13             for (int c = 0; c < n; c++) {
14                 if (dp[r][c] > 0) {
15                     int dirs[4][2] = {{-1,0}, {1,0}, {0,-1}, {0,1}};
16                     for (int d = 0; d < 4; d++) {
17                         int nr = r + dirs[d][0];
18                         int nc = c + dirs[d][1];
19                         if (nr < 0 || nr >= m || nc < 0 || nc >= n) {
20                             count = (count + dp[r][c]) % MOD;
21                         } else {
22                             nextDp[nr][nc] = (nextDp[nr][nc] + dp[r][c]) % MOD;
23                         }
24                     }
25                 }
26             }
27         }
28         memcpy(dp, nextDp, sizeof(dp));
29     }
30     return (int)count;
31 }
32
33 int main() {
34     printf("Result 1: %d\n", findPaths(2, 2, 2, 0, 0)); // Output: 6
35     printf("Result 2: %d\n", findPaths(1, 3, 3, 0, 1)); // Output: 12
36     return 0;
37 }

```

OUTPUT:

```

Result 1: 6
Result 2: 12

```

12. You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

Examples:

(i) Input : nums = [2, 3, 2]

Output : The maximum money you can rob without alerting the police is 3 (robbing house 1).

(ii) Input : nums = [1, 2, 3, 1]

Output : The maximum money you can rob without alerting the police is 4 (robbing house 1 and house 3).

```

main.c
1  #include <stdio.h>
2
3  #define MAX(a, b) ((a) > (b) ? (a) : (b))
4
5  // Helper function to solve the Linear House Robber problem
6  int robLinear(int* nums, int start, int end) {
7      int prev2 = 0;
8      int prev1 = 0;
9
10     for (int i = start; i <= end; i++) {
11         int current = MAX(prev1, prev2 + nums[i]);
12         prev2 = prev1;
13         prev1 = current;
14     }
15     return prev1;
16 }
17
18 int rob(int* nums, int numsSize) {
19     if (numsSize == 0) return 0;
20     if (numsSize == 1) return nums[0];
21
22     // Scenario 1: Rob houses 0 to n-2. Scenario 2: Rob houses 1 to n-1.
23     int max1 = robLinear(nums, 0, numsSize - 2);
24     int max2 = robLinear(nums, 1, numsSize - 1);
25
26     return MAX(max1, max2);
27 }
28
29 int main() {
30     int case1[] = {2, 3, 2};
31     int case2[] = {1, 2, 3, 1};
32
33     printf("Result 1: %d\n", rob(case1, 3)); // Output: 3
34     printf("Result 2: %d\n", rob(case2, 4)); // Output: 4
35     return 0;
36 }

```

OUTPUT:

```

Result 1: 3
Result 2: 4

```

13. You are climbing a staircase. It takes  $n$  steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Examples:

- (i) Input:  $n=4$     Output: 5
- (ii) Input:  $n=3$     Output: 3

```
main.c
1  #include <stdio.h>
2  int climbStairs(int n) {
3      // Base cases
4      if (n <= 0) return 0;
5      if (n == 1) return 1;
6      if (n == 2) return 2;
7      int first = 1;
8      int second = 2;
9      int current = 0;
10
11     for (int i = 3; i <= n; i++) {
12         current = first + second;
13         first = second;
14         second = current;
15     }
16
17     return second;
18 }
19
20 int main() {
21     int n1 = 4;
22     int n2 = 3;
23
24     printf("Input: n = %d | Output: %d\n", n1, climbStairs(n1));
25     printf("Input: n = %d | Output: %d\n", n2, climbStairs(n2));
26     return 0;
27 }
28
```

input

```
Input: n = 4 | Output: 5
Input: n = 3 | Output: 3
```

14. A robot is located at the top-left corner of a  $m \times n$  grid. The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?

Examples:

- (i) Input:  $m=7, n=3$       Output: 28
- (ii) Input:  $m=3, n=2$       Output: 3

```
main.c
1 #include <stdio.h>
2 long long uniquePaths(int m, int n) {
3     int total_steps = m + n - 2;
4
5     int k = (m - 1 < n - 1) ? m - 1 : n - 1;
6
7     long long res = 1;
8
9     for (int i = 1; i <= k; i++) {
10         res = res * (total_steps - k + i) / i;
11     }
12
13     return res;
14 }
15
16 int main() {
17
18     int m1 = 7, n1 = 3;
19     printf("Input: m = %d, n = %d | Output: %lld\n", m1, n1, uniquePaths(m1, n1));
20
21     int m2 = 3, n2 = 2;
22     printf("Input: m = %d, n = %d | Output: %lld\n", m2, n2, uniquePaths(m2, n2));
23
24     return 0;
25 }
26
```

input

Input: m = 7, n = 3 | Output: 28  
Input: m = 3, n = 2 | Output: 3

15. In a string *S* of lowercase letters, these letters form consecutive groups of the same character. For example, a string like *s* = "abbxxxxzzy" has the groups "a", "bb", "xxxx", "z", and "yy". A group is identified by an interval [start, end], where start and end denote the start and end indices (inclusive) of the group. In the above example, "xxxx" has the interval [3,6]. A group is considered large if it has 3 or more characters. Return the intervals of every large group sorted in increasing order by start index.

Example 1:

Input: *s* = "abbxxxxzzy"

Output: [[3,6]]

Explanation: "xxxx" is the only large group with start index 3 and end index 6.

Example 2:

Input: *s* = "abc"

Output: []

Explanation: We have groups "a", "b", and "c", none of which are large groups.

main.c

```
1  #include <stdio.h>
2  #include <string.h>
3  void findLargeGroupIntervals(const char *s) {
4      int len = strlen(s);
5      int start = 0;
6      int found_any = 0;
7      printf("Input: s = \"%s\"\nOutput: [", s);
8      for (int end = 0; end <= len; end++) {
9
10         if (end == len || s[end] != s[start]) {
11             int group_length = end - start;
12
13             if (group_length >= 3) {
14                 if (found_any) {
15                     printf(", ");
16                 }
17                 printf("[%d,%d]", start, end - 1);
18                 found_any = 1;
19             }
20
21             start = end;
22         }
23     }
24     printf("]\n\n");
25 }
26
27 int main() {
28     char s1[] = "abbxxxxzzy";
29     findLargeGroupIntervals(s1);
30     char s2[] = "abc";
31     findLargeGroupIntervals(s2);
32     char s3[] = "aaaabbbcccc";
33     findLargeGroupIntervals(s3);
34
35     return 0;
36 }
```

OUTPUT:

```
Input: s = "abbxxxxzzy"
Output: [[3,6]]

Input: s = "abc"
Output: []

Input: s = "aaaabbbcccc"
Output: [[0,3], [4,6], [7,11]]
```

16."The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970." The board is made up of an  $m \times n$  grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules

```
main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int dirs[8][2] = {
5      {-1, -1}, {-1, 0}, {-1, 1},
6      {0, -1},      {0, 1},
7      {1, -1}, {1, 0}, {1, 1}
8  };
9
10 void gameOfLife(int** board, int boardSize, int* boardColSize) {
11     int m = boardSize;
12     int n = *boardColSize;
13
14     for (int i = 0; i < m; i++) {
15         for (int j = 0; j < n; j++) {
16             int liveNeighbors = 0;
17
18             for (int d = 0; d < 8; d++) {
19                 int ni = i + dirs[d][0];
20                 int nj = j + dirs[d][1];
21
22                 if (ni >= 0 && ni < m && nj >= 0 && nj < n) {
23                     if (board[ni][nj] == 1 || board[ni][nj] == 2) {
24                         liveNeighbors++;
25                     }
26                 }
27             }
28
29             if (board[i][j] == 1) {
30                 if (liveNeighbors < 2 || liveNeighbors > 3) {
31                     board[i][j] = 2;
32                 }
33             } else {
34                 if (liveNeighbors == 3) {
35                     board[i][j] = 1;
36                 }
37             }
38         }
39     }
40 }
```

```

38         board[i][j] = 3;
39     }
40 }
41 }
42 }
43
44 for (int i = 0; i < m; i++) {
45     for (int j = 0; j < n; j++) {
46         if (board[i][j] == 2) {
47             board[i][j] = 0;
48         } else if (board[i][j] == 3) {
49             board[i][j] = 1;
50         }
51     }
52 }
53 }
54 void printBoard(int** board, int m, int n) {
55     for (int i = 0; i < m; i++) {
56         for (int j = 0; j < n; j++) {
57             printf("%d ", board[i][j]);
58         }
59         printf("\n");
60     }
61     printf("\n");
62 }
63
64 int main() {
65     int m = 4;
66     int n = 3;
67
68     // Test case initialization
69     int initial_values[4][3] = {
70         {0, 1, 0},
71         {0, 0, 1},
72         {1, 1, 1},
73         {0, 0, 0}
74 };

```

```

75
76 int** board = (int**)malloc(m * sizeof(int*));
77 for (int i = 0; i < m; i++) {
78     board[i] = (int*)malloc(n * sizeof(int));
79     for (int j = 0; j < n; j++) {
80         board[i][j] = initial_values[i][j];
81     }
82 }
83
84 int boardColSize = n;
85
86 printf("Initial Board Configuration:\n");
87 printBoard(board, m, n);
88
89 gameOfLife(board, m, &boardColSize);
90
91 printf("Next Generation State Configuration:\n");
92 printBoard(board, m, n);
93
94 for (int i = 0; i < m; i++) {
95     free(board[i]);
96 }
97 free(board);
98
99 return 0;
100 }

```

OUTPUT:

```
Initial Board Configuration:
0 1 0
0 0 1
1 1 1
0 0 0

Next Generation State Configuration:
0 0 0
1 0 1
0 1 1
0 1 0
```